Java Backend Architecture

Interview Series

Chapter 15.1
Monolith vs Microservices vs Modular Monolith

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 15.1 – Monolith vs Microservices vs Modular Monolith

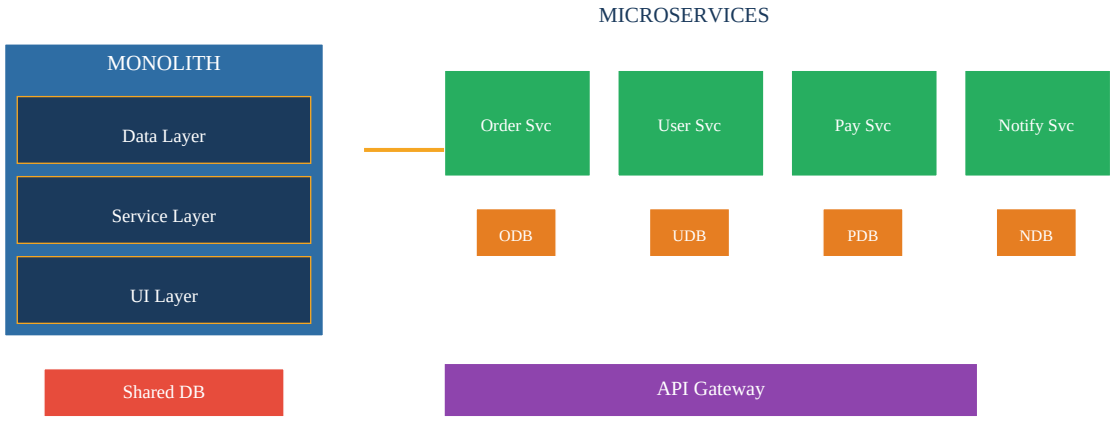
Introduction

Choosing the right architectural style is one of the most consequential decisions a backend team can make. Monoliths, microservices, and modular monoliths each represent distinct trade-off points on the axes of operational complexity, team autonomy, deployment independence, and development velocity. Understanding when to apply each—and how to migrate between them—is a core senior-engineer skill.

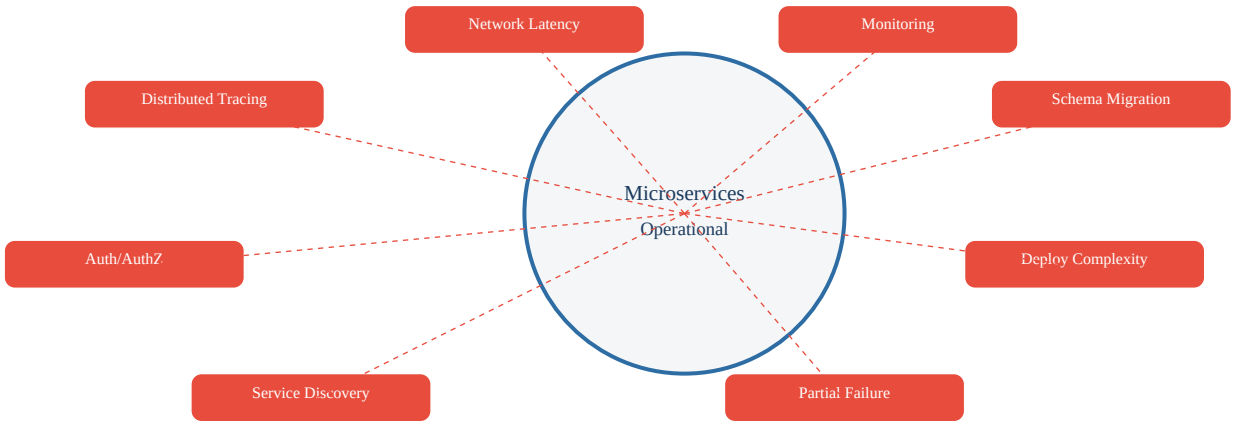
Key Definitions

Monolith	Single deployable unit where all modules share one process, one database, and are deployed together. Simple to develop and debug early; risks becoming a "big ball of mud" as it grows.
Microservices	Suite of small, independently deployable services each owning its data store, communicating over network. Enables team autonomy and polyglot stacks; adds significant operational overhead.
Modular Monolith	Single deployable unit with strict, enforced module boundaries (Java 9 modules or ArchUnit). Shared DB but module-owned tables. Best of both worlds for small-to-mid teams (<50 engineers).
DB per Service	Each microservice owns its persistence — no shared schema. Enforces loose coupling at data level; requires patterns like Saga/CQRS for cross-service queries.
Big Ball of Mud	Monolith that accumulated uncontrolled dependencies: any module can call any other, circular imports, no clear ownership. The result of missing modular discipline.
ArchUnit	Java test library to enforce architecture rules (e.g., "order module must not import payment module") as unit tests, preventing boundary violations at build time.
Strangler Fig	Incremental migration pattern: new functionality is built as microservices behind a facade/proxy while legacy monolith routes continue to function until replaced.
Conway's Law	System architecture mirrors communication structure of the organisation. Microservices work best when team topology aligns with service ownership.

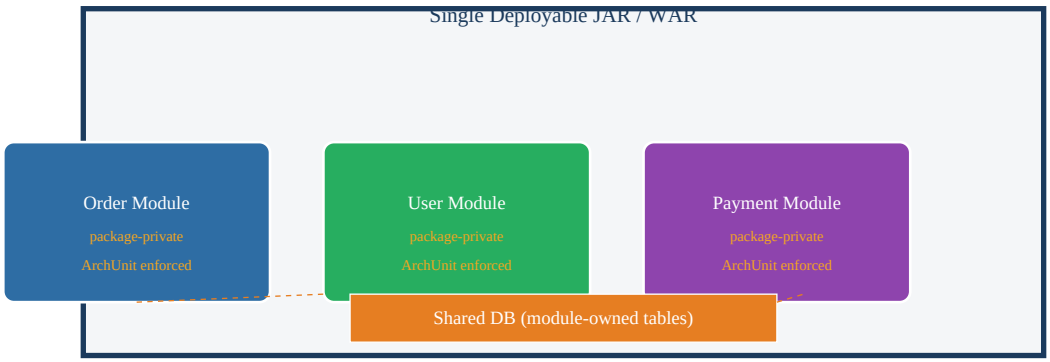
Architecture Diagram – Three Styles Compared



Microservices Operational Complexity



Modular Monolith – Module Boundary Enforcement



Comparison Table

Dimension	Monolith	Microservices	Modular Monolith
Deploy unit	One JAR/WAR	N independent services	One JAR/WAR
Database	Shared DB	DB per service	Shared DB (module tables)

Team size	Any (small best)	Large (>50)	<50 engineers
Operational complexity	Low	High	Low
Independent scaling	No	Yes per service	No
Polyglot stacks	No	Yes	No (JVM)
Network failures	None internally	Frequent concern	None internally
Distributed tracing	Not needed	Essential	Not needed
Boundary enforcement	Convention only	Network enforced	ArchUnit / JPMS
Good starting point?	Yes – always start here	Rarely greenfield	Yes – with discipline

Spring Boot Modular Monolith – ArchUnit Enforcement

```
// ArchUnit rule: order module must not import payment internals @AnalyzeClasses(packages =
"com.example") public class ArchitectureTest { @ArchTest static final ArchRule
order_must_not_depend_on_payment_internals = noClasses().that().resideInAPackage("..order..")
.should().dependOnClassesThat() .resideInAPackage("..payment.internal.."); @ArchTest static
final ArchRule no_cycles = slices().matching("com.example.(*)..").should().beFreeOfCycles(); }
```

Microservices – Spring Boot Multi-Module Setup

```
# docker-compose.yml – independent service deployments services: order-service: image:
myapp/order-service:1.2.0 ports: ["8081:8080"] environment: SPRING_DATASOURCE_URL:
jdbc:postgresql://order-db:5432/orders user-service: image: myapp/user-service:2.0.1 ports:
["8082:8080"] environment: SPRING_DATASOURCE_URL: jdbc:postgresql://user-db:5432/users
api-gateway: image: myapp/gateway:1.0.0 ports: ["8080:8080"]
```

Migration Strategy – Strangler Fig Pattern

```
// Spring Cloud Gateway – strangler fig routing @Configuration public class GatewayConfig {
@Bean public RouteLocator routes(RouteLocatorBuilder b) { return b.routes() // New microservice
handles /api/orders .route("orders-new", r -> r.path("/api/orders/**")
.uri("lb://order-service")) // Legacy monolith still handles everything else .route("legacy", r
-> r.path("/**") .uri("http://legacy-monolith:8080")) .build(); } }
```

Interview Q&A;

Q1. What are the three main architectural styles for backend systems?

Monolith (single deployable, shared DB), Microservices (independent services, DB per service, polyglot), and Modular Monolith (single deploy with enforced module boundaries, module-owned tables). Each solves different scale/team problems.

Q2. When should you NOT use microservices?

Avoid microservices when: (1) team is small (<10 engineers), (2) domain is not well understood—premature decomposition creates wrong service boundaries, (3) operational maturity is low—no k8s/CI/CD/observability stack, (4) starting a greenfield project—always start as modular monolith.

Q3. What is the 'big ball of mud' anti-pattern?

A monolith that accumulated unrestricted cross-module dependencies: any class imports any other, circular imports, no clear ownership. Results from missing architectural discipline. Prevented by ArchUnit rules enforced in CI.

Q4. What does 'DB per service' mean and why is it important?

Each microservice owns its own schema and database—no other service can directly query or write to it. This enforces loose coupling at the data level. Services must expose APIs for data access. Downside: cross-service queries require Saga/CQRS patterns and eventual consistency.

Q5. How does a Modular Monolith differ from a plain Monolith?

A Modular Monolith has explicitly enforced boundaries between modules—via Java 9 modules (module-info.java) or package-private access + ArchUnit tests. Module A cannot directly call Module B's internal classes. This prevents the big ball of mud while keeping single-deploy simplicity.

Q6. What is Conway's Law and how does it affect service decomposition?

Conway's Law: 'Organizations design systems that mirror their own communication structure.' If you have one team, you'll build a monolith; two teams naturally create two services. Microservices succeed when team topology aligns with service ownership—each team fully owns one or more services.

Q7. Explain the Strangler Fig migration pattern.

Named after a fig tree that grows around a host and eventually replaces it. A proxy/API gateway sits in front of the legacy monolith. New features are built as microservices; traffic for those features is routed to new services. Legacy routes remain until fully replaced. Risk-free incremental migration with no big-bang rewrite.

Q8. What is distributed tracing and why is it essential for microservices?

In microservices a single user request may traverse 5-10 services. Distributed tracing propagates a correlation ID (trace ID) across all service calls so you can reconstruct the full request journey. Tools: Micrometer Tracing + Zipkin/Jaeger in Spring Boot. Without it, debugging cross-service latency is nearly impossible.

Q9. What is the recommended starting architecture for a new product?

Start with a well-structured Modular Monolith. Domain boundaries are not fully understood early; premature microservices decomposition causes wrong boundaries that are expensive to fix. Extract services only when a module needs independent scaling, polyglot tech, or separate team ownership.

Q10. How do you enforce module boundaries in a Java Modular Monolith without Java 9 modules?

Use ArchUnit: write @ArchTest rules that run in CI. E.g., 'no class in package order.* may depend on payment.internal.*'. Combined with package-private visibility (classes not exposed outside their package), this prevents unauthorized access.

Q11. What operational challenges do microservices introduce?

Network latency and partial failures, distributed tracing across services, service discovery (Consul/k8s DNS), distributed authentication (JWT propagation), schema migration per service, independent deployment pipelines, centralized logging aggregation (ELK/Loki), distributed transactions (Saga pattern).

Q12. What is polyglot persistence in microservices?

Each service can use the database technology best suited to its needs: Order service uses PostgreSQL, Product catalog uses Elasticsearch, Session service uses Redis, Recommendation service uses Neo4j. Possible because services are isolated—no shared schema coupling forces a single DB choice.

Q13. What is an API Gateway and why is it needed in microservices?

A single entry point for all client requests. Handles: routing to backend services, authentication/authorization, rate limiting, SSL termination, request aggregation (BFF pattern), protocol translation. Spring Cloud Gateway or AWS API Gateway are common choices.

Q14. Describe the parallel run migration strategy.

Both old monolith and new microservice handle the same traffic simultaneously. Results are compared. If the new service produces identical output, traffic is switched. Used for high-risk migrations where correctness must be proven before cutover. Feature flags control which path is active.

Q15. How do microservices handle service failures differently from monoliths?

In a monolith, a failure crashes the whole process (all-or-nothing). In microservices, a downstream service failure should be isolated using Circuit Breaker (Resilience4j), Bulkhead (separate thread pools), and Timeout patterns. Properly implemented, a payment service failure should not crash the catalog service.

Q16. What is the difference between horizontal and vertical decomposition?

Horizontal (technical layer): UI layer, service layer, data layer—fragile, changes touch all layers. Vertical (business capability): Order service, User service, Payment service—each owns full stack for its domain. DDD recommends vertical/business-capability decomposition for stable service boundaries.

Q17. How do you handle cross-cutting concerns (logging, auth, tracing) across microservices?

Use a sidecar pattern (Istio/Envoy) or Spring Boot AutoConfiguration shared libraries. Centralized logging with correlation IDs (MDC in SLF4J), JWT token propagation via RestTemplate/WebClient interceptors, distributed tracing via Micrometer Tracing. Avoid duplicating this logic in every service.

Q18. What is the two-pizza team rule and how does it relate to microservices?

Amazon's guideline: a service team should be small enough to feed with two pizzas (~6-8 people). Each team owns one or a few microservices end-to-end (build, deploy, operate). Large teams indicate a service is too big and should be split. Team size drives service granularity.

Q19. What risks come with premature microservices extraction?

Wrong service boundaries (discovered only after deep domain understanding), distributed monolith (services too tightly coupled—shared DB, sync chains), high operational overhead before product-market fit, slow feature development due to cross-service coordination, and difficulty refactoring boundaries once established.

Q20. How does Spring Boot support modular development?

Spring Boot works well with Maven/Gradle multi-module projects. Each module is a separate JAR with its own Spring configuration. Boundaries are enforced via ArchUnit. In a modular monolith, all modules are packaged into one deployable; in microservices, each module gets its own Spring Boot application and Docker image.

Q21. What is the role of CI/CD in microservices?

Each service needs its own independent CI/CD pipeline: test → build Docker image → push to registry → deploy to staging → canary release → production. With 50 services this is 50 independent pipelines. Tools: GitHub Actions, Jenkins, Tekton. Without mature CI/CD, microservices become a deployment nightmare.

Q22. Explain the concept of service mesh.

A dedicated infrastructure layer (Istio, Linkerd) that handles service-to-service communication: mTLS encryption, load balancing, circuit breaking, distributed tracing, retries—all without code changes. Implemented as sidecar proxies (Envoy) injected into each service pod. Useful for large microservice deployments.

Q23. What is the difference between integration testing and contract testing in microservices?

Integration testing: spin up real service + real dependencies (Testcontainers), test interactions. Slow, brittle. Contract testing (Pact): consumer defines expected request/response; provider verifies it matches in CI. Fast, no need to run all services together. Catches breaking API changes before deployment.

Q24. How do you manage configuration across multiple microservices?

Spring Cloud Config Server: centralized Git-backed configuration. Each service fetches its config on startup. Environment-specific profiles (dev/staging/prod). Alternatively, Kubernetes ConfigMaps/Secrets. Never hardcode URLs or credentials—use externalized config per 12-Factor App principles.

Q25. What metrics should you monitor per microservice?

RED metrics: Rate (requests/sec), Errors (error rate %), Duration (latency p50/p95/p99). USE metrics for infrastructure: Utilization, Saturation, Errors. Business metrics: orders processed, payment success rate. Tools: Micrometer + Prometheus + Grafana.

Q26. When is a monolith the right choice?

Monolith is right when: team <10, domain not fully understood, early-stage product needing fast iteration, no dedicated DevOps/SRE capability, simple deployment requirements. Most successful products started as monoliths (GitHub, Shopify, Stack Overflow) and extracted services only when specific scale needs demanded it.

Q27. What is the 'modular monolith first' strategy?

Build as a modular monolith with strict boundaries enforced by ArchUnit. When a module needs independent scaling or separate team ownership, extract it to a microservice. The module boundaries become the service boundaries—much cleaner extraction than carving up an unstructured monolith.

Q28. How do you handle database migrations in microservices?

Each service manages its own schema via Flyway or Liquibase. Migrations run on service startup. Schema changes must be backward-compatible (additive): add columns with defaults, never remove/rename columns without a multi-step migration. Blue-green deployments require two schema versions to be compatible simultaneously.

Q29. What is backward-compatible schema migration?

A migration that old and new versions of a service can both work with. Rule: always add columns with defaults, never remove or rename in the same deployment. Multi-step: (1) add new column, (2) dual-write old+new, (3) backfill, (4) switch reads to new, (5) drop old column in later release.

Q30. How does health checking work in microservices?

Spring Boot Actuator /actuator/health endpoint returns UP/DOWN. Kubernetes uses liveness probes (restart on failure) and readiness probes (remove from load balancer until ready). Custom health indicators can check DB connections, Kafka connectivity, downstream service health.

Q31. What is the difference between liveness and readiness probes?

Liveness probe: is the container alive? If not, Kubernetes restarts it. Readiness probe: is the container ready to serve traffic? If not, removed from Service endpoints. A warming-up service should fail readiness but pass liveness. A deadlocked service should fail liveness to trigger restart.

Q32. Explain blue-green deployment in the context of microservices.

Two identical production environments (blue = current, green = new version). Deploy new version to green, run smoke tests, then switch load balancer to green. Instant rollback by switching back to blue. Each microservice can do independent blue-green deployments on its own schedule.

Q33. What is a canary release?

Gradually route a small percentage of traffic (1-5%) to the new version. Monitor error rates and latency. If healthy, increase traffic incrementally. Rollback by routing all traffic back to old version. Lower risk than blue-green but slower. Kubernetes supports this natively via multiple Deployments + Service weights.

Q34. How do you handle service discovery in microservices?

Two approaches: (1) Client-side discovery: client queries registry (Eureka/Consul), gets instance list, load-balances. Spring Cloud LoadBalancer supports this. (2) Server-side discovery: DNS-based (Kubernetes Services). Kubernetes is now the standard: each service gets a DNS name (my-service.namespace.svc.cluster.local).

Q35. What is the difference between synchronous and asynchronous microservice communication?

Synchronous (REST/gRPC): caller waits for response—simple, immediate feedback, but couples availability. Asynchronous (Kafka/RabbitMQ): caller publishes event, continues—decoupled availability, higher throughput, but eventual consistency and harder debugging. Use async for operations that don't need immediate response.

Q36. How do you size microservices?

Rule: a service should be owned by one team and deployable independently. Size by business capability, not technical layer. A service should have a clear, bounded domain with its own data. Too fine-grained (nano-services) causes chatty communication; too coarse-grained defeats the purpose of microservices.

Q37. What is the sidecar pattern?

Deploy a secondary container alongside the main service container in the same pod. The sidecar handles cross-cutting concerns: Envoy proxy for traffic management, Filebeat for log shipping, Vault agent for secrets. Decouples infrastructure concerns from application code.

Q38. What is an anti-corruption layer (ACL)?

A translation layer between two bounded contexts with different domain models. Prevents the concepts of one context from 'corrupting' (leaking into) another. E.g., if integrating with a legacy CRM that uses 'account' for what your domain calls 'customer', the ACL translates between the two models.

Q39. Describe the decomposition by business capability approach.

Identify stable business capabilities (Order Management, Customer Management, Payment Processing) that change independently. Each becomes a service boundary. Business capabilities are stable—they change less often than technical implementations. This produces service boundaries that survive organizational and technical change.

Q40. What is shared kernel in DDD context maps?

A subset of the domain model shared between two bounded contexts—both teams must agree on changes. Suitable for very tightly coupled contexts that share core concepts. High coordination overhead. In microservices, often implemented as a shared library JAR, but must be versioned carefully.

Q41. How do you handle authentication across microservices?

Use JWT (JSON Web Token): API Gateway validates the JWT and forwards it to downstream services. Each service validates the token signature (public key) without calling a central auth server. Service-to-service calls use separate service account tokens (OAuth2 Client Credentials flow).

Q42. What is the Open-Host Service pattern?

A bounded context that defines a formal, published protocol (API) for integration with multiple consumers. The service team maintains backward compatibility and versioning. Consumers integrate via the published protocol without the service needing custom integration for each consumer. Spring REST API with OpenAPI spec is an example.

Q43. Explain Published Language pattern.

A well-documented, shared data format used for communication between bounded contexts. Examples: JSON Schema, Avro schema in Schema Registry, Protobuf .proto files. Allows contexts to evolve independently as long as they conform to the published schema.

Q44. How does Java 9 module system (JPMS) enforce boundaries in a modular monolith?

module-info.java declares: (1) exports—which packages are public API, (2) requires—which other modules are needed, (3) opens—which packages are open for reflection. The JVM enforces these at runtime and compile time. module-info.java com.example.order { exports com.example.order.api; requires com.example.user.api; }

Q45. What are the signs that a modular monolith should be split into microservices?

(1) A specific module needs independent scaling (payment processing vs. browsing), (2) Different teams need to deploy independently without coordination, (3) Module needs a different tech stack (ML model serving needs Python), (4) Module has significantly different reliability/SLA requirements, (5) Team size has grown beyond ~50 engineers.

Q46. How do you implement distributed tracing in Spring Boot?

Add Micrometer Tracing + Brave/OTel bridge. Configure Zipkin/Jaeger exporter. Spring Boot auto-configures trace ID propagation via HTTP headers (traceparent/b3). All log statements automatically include trace ID via MDC. spring.sleuth.sampler.probability=1.0 for full sampling in dev.

Q47. What is the difference between a monorepo and polyrepo for microservices?

Monorepo: all services in one Git repository. Benefits: atomic cross-service changes, shared tooling, easier refactoring. Downsides: CI times grow, requires careful tooling (Nx, Bazel, Gradle included builds). Polyrepo: one repo per service. Benefits: full independence. Downsides: cross-service changes need multiple PRs.

Q48. Describe a real-world microservices failure pattern you should avoid.

The 'distributed monolith': services that look microservices but are tightly coupled—they share a database, make synchronous calls in long chains (A calls B calls C calls D), and must be deployed together. This combines the worst of both worlds: operational complexity of microservices + coupling of a monolith. Prevention: enforce DB-per-service, avoid sync call chains, use events for cross-service communication.

Q49. How do you test a modular monolith effectively?

Unit tests per module with mocked inter-module interfaces. Integration tests using Spring Boot test slices @SpringBootTest with Testcontainers for real DB. ArchUnit tests in CI to enforce boundary rules. End-to-end tests for critical user journeys. The single-JVM nature makes integration testing faster than microservices.

Q50. What is the difference between functional and non-functional decomposition?

Functional decomposition: split by use case/business capability (Order, Payment, Catalog). Non-functional decomposition: split by performance/scalability needs (high-throughput event processing service vs. low-traffic admin service). Best practice: decompose functionally first, then further split if non-functional requirements diverge significantly.

Q51. What is the typical migration path from monolith to microservices?

(1) Add module boundaries to monolith (ArchUnit), (2) Identify extraction candidates (high load, separate team, different tech), (3) Add Strangler Fig proxy in front of monolith, (4) Extract first service (start with least coupled), (5) Route traffic via proxy, (6) Verify parity with parallel run, (7) Decommission monolith code for that capability, (8) Repeat for remaining modules.

Q52. Explain the 12-Factor App principles in the context of microservices.

Key factors: (I) Codebase—one codebase per service in version control. (III) Config—externalize all config (no hardcoded URLs). (IV) Backing services—treat DB/cache as attached resources. (VI) Processes—stateless services, store state in backing services. (IX) Disposability—fast startup, graceful shutdown. (XI) Logs—treat as event streams to stdout, aggregate centrally.